

Unit 7: Object-Oriented Development 8 LHs

By: Kabita Dhital

BIM 5th Sem

Introduction to Object-Oriented Development

- Object-oriented design (OOD) is the philosophy of developing an object-oriented model of a software system, by defining the classes and their interactions with one another.
- **Object-Oriented Development (OOD)** is a software development approach that focuses on designing and structuring software based on objects, which represent real-world entities by encapsulating both data (attributes) and behavior (methods).
- The major benefits of OOD are easy maintainable systems, easy understandable systems, easy expandable systems, and reusable components of a system.
- This paradigm enhances modularity, reusability, and maintainability in software design by organizing code into self-contained objects that interact with one another.

Key Concepts of Object-Oriented Development:

1.) Objects:

- Objects are instances of classes that represent real-world entities with attributes (data) and methods (behavior).
- **Example:** A Car object may have attributes like color, speed, and brand, with methods like `accelerate()` and `brake()`.

2.) Classes:

- A class is a blueprint or template used to create objects. It defines the attributes and methods that its objects will have.
- A class is a user-defined data type.
- It consists of data members and member functions, which can be accessed and used by creating an instance of that class.
- Example: A Car class may define properties (model, year) and methods (`start()`, `stop()`).
- **For Example:** Consider the Class of Cars. There may be many cars with different names and brands but all of them will share some common properties like all of them will have 4 wheels, Speed Limit, Mileage range, etc. So here, Car is the class, and wheels, speed limits, mileage are their properties.

3.) Encapsulation

- Encapsulation is the practice of bundling data and methods within an object while restricting access to certain details.
- Example: A BankAccount class may hide balance details using private access modifiers and provide `deposit()` and `withdraw()` methods for controlled access.

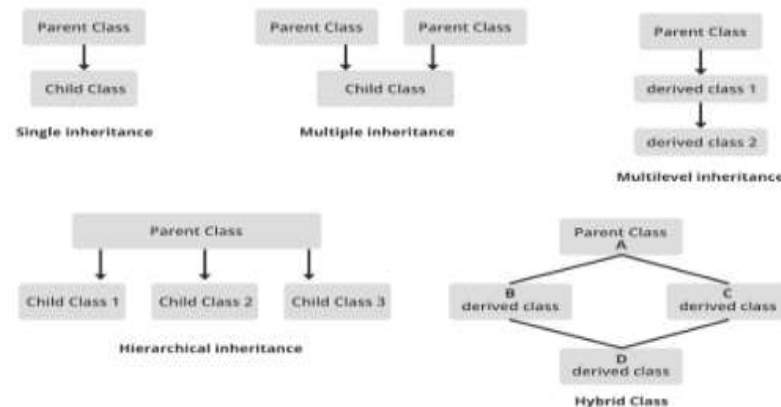
Key Concepts of Object-Oriented Development:



Consider a real-life example of encapsulation, in a company, there are different sections like the accounts section, finance section, sales section, etc. The finance section handles all the financial transactions and keeps records of all the data related to finance. Similarly, the sales section handles all the sales-related activities and keeps records of all the sales.

4.) Inheritance:

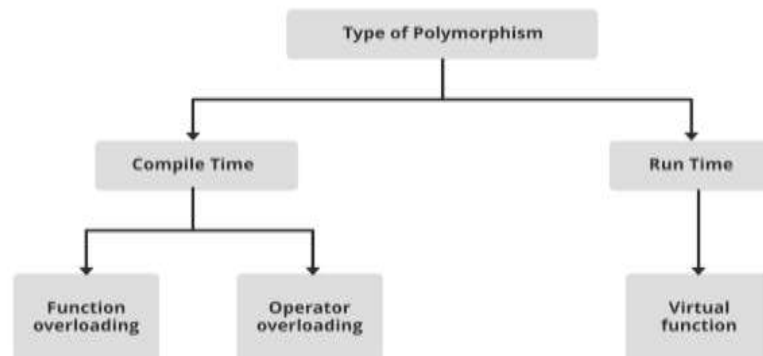
- Inheritance allows a child class to derive properties and behavior from a parent class, promoting code reuse. When we write a class, we inherit properties from other classes.
- Example: A Truck class can inherit from a Vehicle class, inheriting attributes like engine and wheels, while adding its own attributes like cargoCapacity



Key Concepts of Object-Oriented Development:

5.) Polymorphism:

- Polymorphism enables an object to take multiple forms by overriding or overloading methods. The word polymorphism means having many forms. In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form. For example, A person at the same time can have different characteristics. Like a man at the same time is a father, a husband, an employee. So the same person possesses different behavior in different situations. This is called polymorphism
- Example: A draw() method in a Shape class can be overridden in subclasses like Circle and Rectangle to provide different drawing implementations.



Key Concepts of Object-Oriented Development:

6.) Abstraction:

- Abstraction hides complex implementation details and exposes only essential functionalities.
- Example: A Payment class may have an abstract method processPayment(), with subclasses CreditCardPayment and PayPalPayment implementing it differently.
- Data abstraction is one of the most essential and important features of object-oriented programming.
- Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation. Consider a real-life example of a man driving a car.
- The man only knows that pressing the accelerators will increase the speed of the car or applying brakes will stop the car, but he does not know about how on pressing the accelerator the speed is increasing, he does not know about the inner mechanism of the car or the implementation of the accelerator, brakes, etc in the car. This is what abstraction is.

Unified Modeling Language

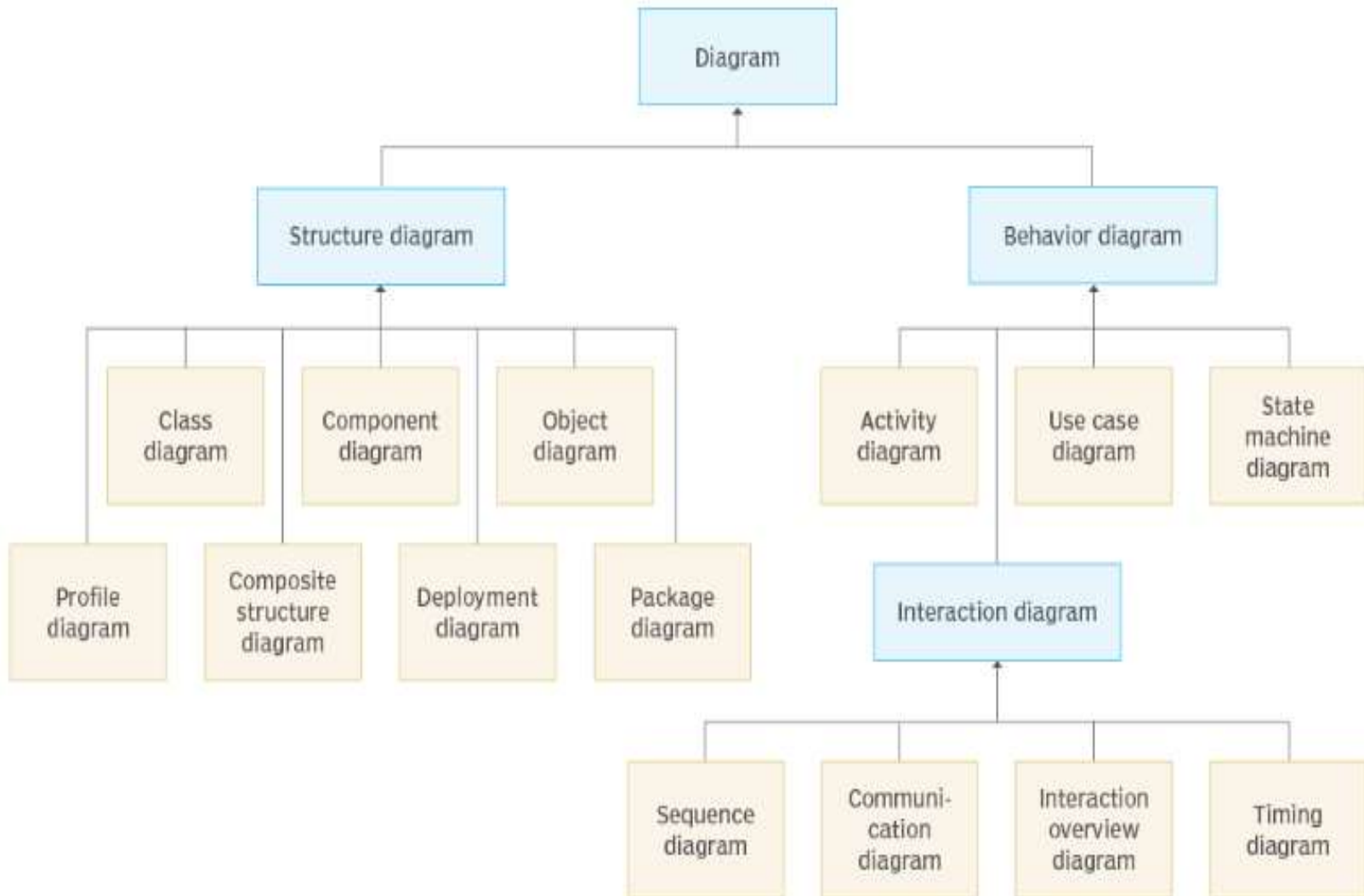
- Unified Modeling Language (UML) is a standardized, visual modeling language used in software engineering to design, visualize, and document software systems and business processes, acting as a blueprint for development.
- It provides a common language for developers and stakeholders to represent structure and behavior, enhancing communication, reducing complexity, and improving efficiency throughout the development lifecycle.
- It helps in visualizing how different parts of a system are structured and how they interact with each other. UML is not a programming language, but a way to create diagrams similar to engineering blueprints. It is widely used for modeling, designing, and analyzing software systems.
- Visualization: Helps represent system structure and behavior clearly.
- Standardization: Provides a common language for developers, architects, and stakeholders.
- Better Understanding: Makes complex systems easier to design and communicate.
- ***Example:*** In an e-commerce system, a UML diagram can show components like User, Product, and Order, and how they interact during processes like placing an order.

Unified Modeling Language

Need of UML

- UML is essential for clearly visualizing and communicating system design among different stakeholders involved in a project.
- Complex applications need collaboration and planning from multiple teams, so a clear and concise way of communication is required.
- Business stakeholders may not understand code, so UML helps explain system requirements, functionalities, and processes in a simple visual way.
- It saves time by allowing teams to visualize workflows, user interactions, and system structure before actual development.
- Provide users with a ready-to-use, expressive visual modeling language so they can develop and exchange meaningful models.
- Provide extensibility and specialization mechanisms to extend the core concepts.
- Be independent of particular programming languages and development processes.
- Provide a formal basis for understanding the modeling language.
- Encourage the growth of the OO tools market.
- Support higher-level development concepts such as collaborations, frameworks, patterns and components.
- Integrate best practices.

Unified Modeling Language



Unified Modeling Language

- Structural diagrams represent the static structure of a system's entities.
- The diagrams are not concerned with time or dynamic behavior, but rather with the underlying entity structure and the relationships between those entities.
- The UML standard defines the following seven types of structural diagrams:
 1. **Profile.** Diagram of the how the system is extended through profiles, which include custom stereotypes, tagged values and constraints.
 2. **Composite structure.** Diagram of the internal structure of a class, its parts and connectors, and its interaction points, both within the class and with other classes.
 3. **Deployment.** Diagram of the distribution of a system's software artifacts across the physical hardware on which the artifacts will be deployed.
 4. **Package.** Diagram of the system's packages -- groupings of UML elements -- and the dependencies between them.
 5. **Class.** Diagram of the system's structure as determined by its classes, their attributes and methods, and the relationships between classes.
 6. **Component.** Diagram of the system's components, such as libraries and executables, and the relationships between those components.
 7. **Object.** Diagram of the system's class structure at a given point in time, thereby providing a snapshot of the class instances, their data structures and the relationships between those instances.

Unified Modeling Language

Behavioral diagrams are more concerned with the system's dynamic behavior.

They take into account how entities change over time and interact with each other when carrying out an action.

The UML standard defines three core behavioral diagrams:

1. **Activity.** Activity diagrams outline the system's operational workflow, rendered in a flow chart that shows the activity carried out by the system's components.
2. **Use case.** Use case diagrams address the system's functionality and how actors interact with the system, based on specific use cases.
3. **State machine.** State machine diagrams cover an element's state during its lifetime as it participates in an activity, thus providing a dynamic model of its behavior.

The UML standard also defines a group of interaction diagrams that are a subset of the behavioral diagrams. The interaction subset includes the following four diagrams:

1. **Sequence.** Diagram of the sequence of interactions between the objects participating in an activity in order to provide a mapping of their communication and message flow.
2. **Communication.** Diagram of the collaboration between objects participating in an activity, similar to a sequence diagram, but focused more on the interactions themselves than on their sequence.
3. **Interaction overview.** Diagram of the control flow of interactions between system entities, thereby providing a high-level view of complex interactions.
4. **Timing.** Diagram of the sequence and timing of events within a given time frame, based on object behavior and interaction.

Activity diagrams

- An activity diagram is a UML behavioral diagram that acts as a sophisticated flowchart, modeling the **dynamic**, step-by-step workflow of a system or business process.
- Activity diagrams show the steps involved in how a system works, helping us understand the flow of control.
- It illustrates the flow from one activity to another, showcasing decision paths, sequential operations, and concurrent, parallel processing.
- Used in both business and software modeling, it helps stakeholders understand system behavior.

Key Purposes and Uses

- **Workflow Modeling:** Visualizes complex business processes or user interactions with a system.
- **Behavioral Modeling:** Captures the dynamic behavior of a system, focusing on action sequences.
- **Use Case Analysis:** Details the steps and exceptions within a specific use case.
- **System Design:** Guides developers by mapping out operational steps for complex algorithms.

Activity diagrams

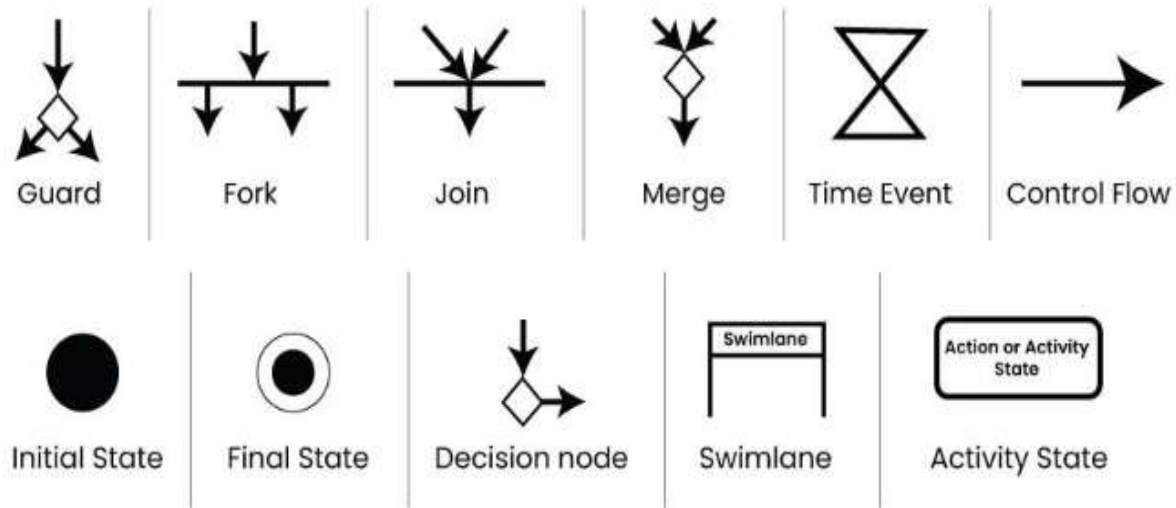
Benefits

- **Clarifies Complex Processes:** Simplifies complex workflows, making them easy to understand, including exceptions and repetitions.
- **Identifies Bottlenecks:** Helps analyze business processes to optimize performance.
- **Enhances Collaboration:** Bridges the gap between business requirements and technical implementation.

Key Components

- **Initial State:** A filled black circle representing the entry point. The starting state before an activity takes place is depicted using the initial state.
- **Activity/Action:** A rounded rectangle representing a single step or action.
- **Control Flow:** An arrow showing the sequence/flow from one action to the next.
- **Decision Node:** A diamond shape that branches the flow based on a condition.
- **Fork/Join:** A bar that splits a task into concurrent flows (fork) or merges parallel paths back together (join).
- **Final State:** An encircled black dot (bullseye) representing the end of the activity.

Activity diagrams



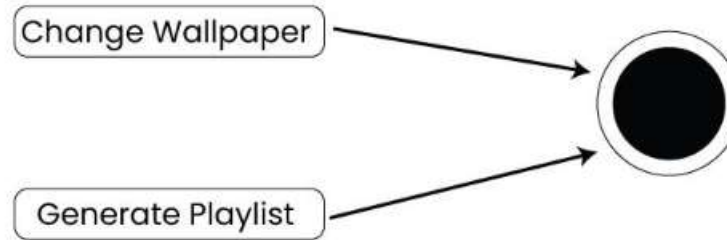
Initial State symbol being used



Activity diagrams

Here both the states transit into one final state using action flow symbols i.e. arrows.

Using Action Flows for Transitions



Activity diagrams

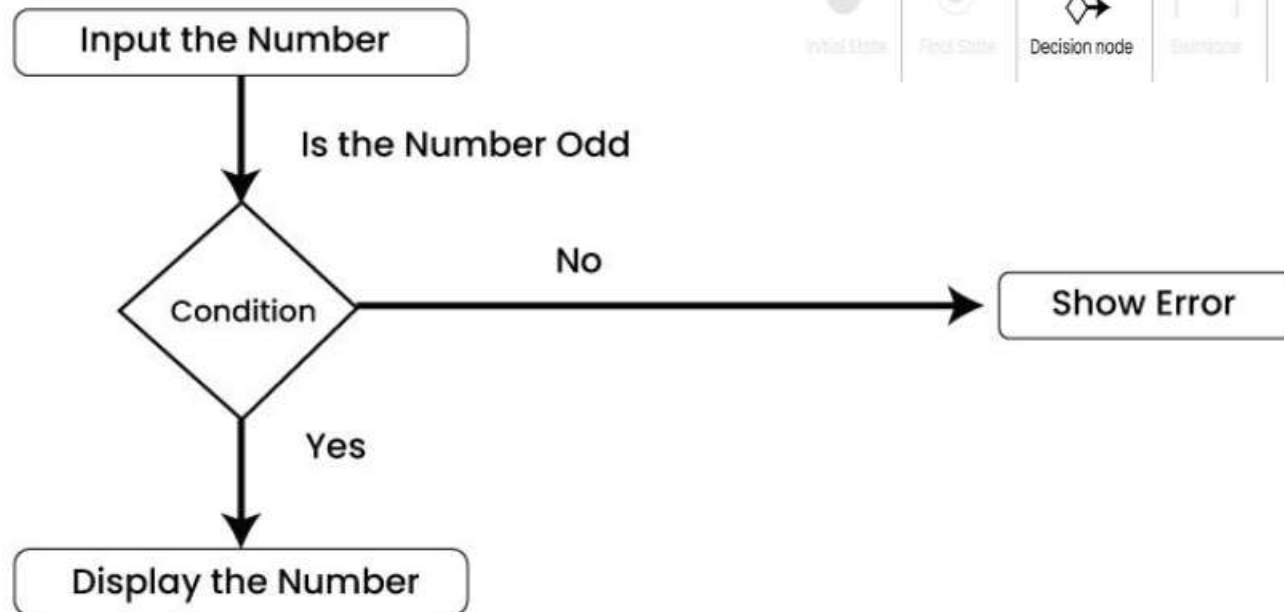
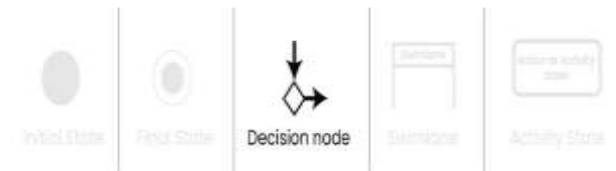
For example:

We apply the conditions on input number to display the result :

- *If number is odd then display the number.*
- *If number is even then display the error.*

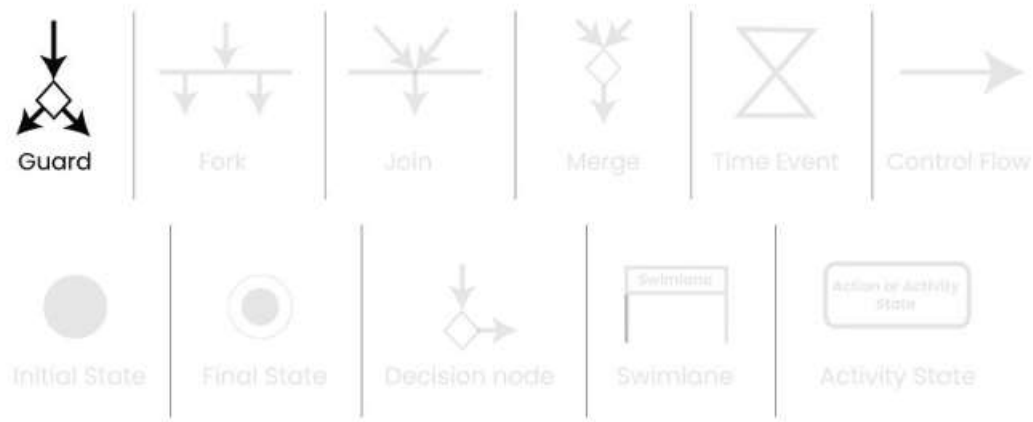


An Activity Diagram using Decision Node



Activity diagrams

A Guard refers to a statement written next to a decision node on an arrow sometimes within square brackets.



Decision Node

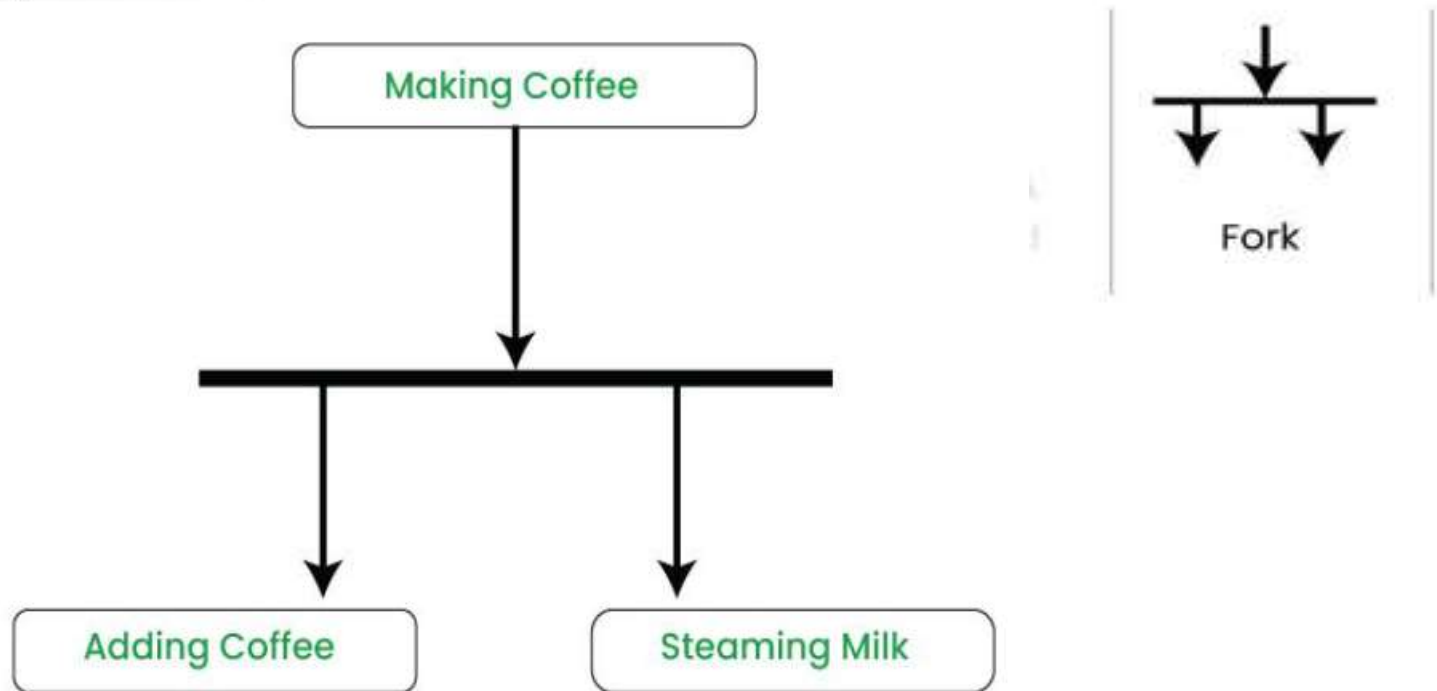
Represent a test condition to ensure that the control flow or object flow only goes down one path



Activity diagrams

- Fork nodes are used to support concurrent activities.
- When we use a fork node when both the activities get executed concurrently i.e. no decision is made before splitting the activity into two parts.
- Both parts need to be executed in case of a fork statement. We use a rounded solid rectangular bar to represent a Fork notation with incoming arrow from the parent activity state and outgoing arrows towards the newly created activities.

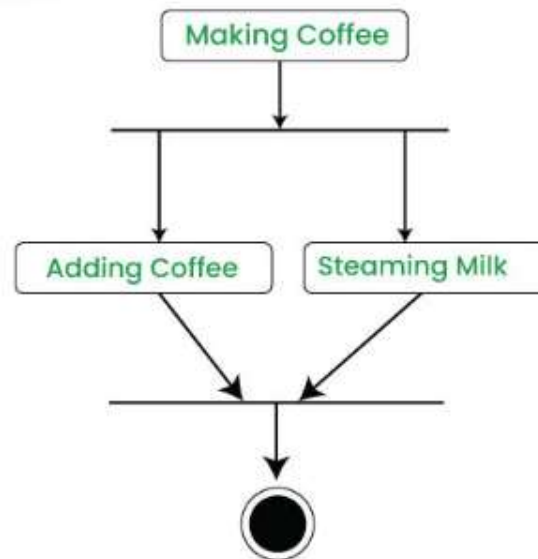
A Diagram using Fork Notation



Activity diagrams

- Join nodes are used to support concurrent activities converging into one. For join notations we have two or more incoming edges and one outgoing edge.

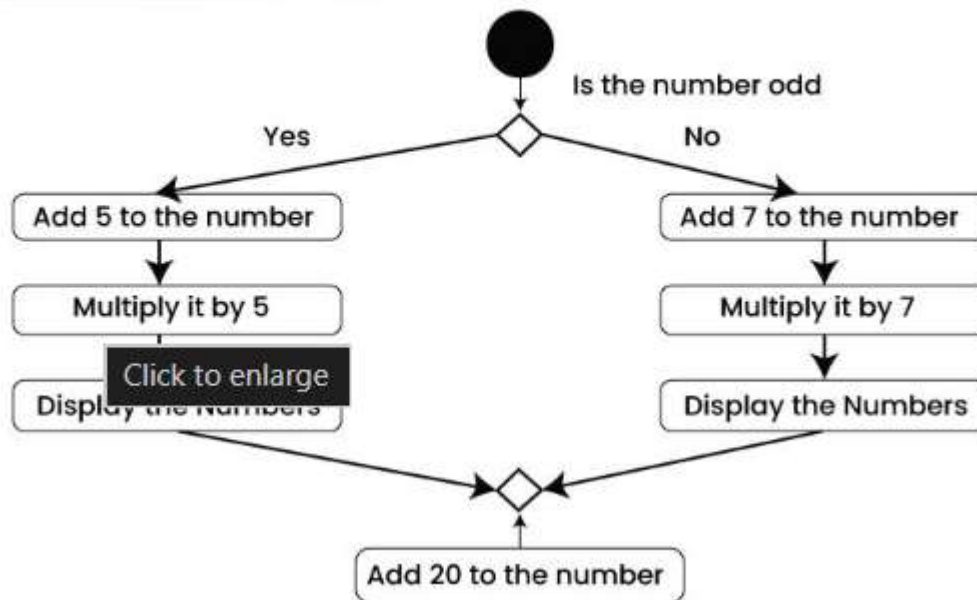
A Diagram using Join Notation



Activity diagrams

- Scenarios arise when activities which are not being executed concurrently have to be merged.
- We use the merge notation for such scenarios.
- We can merge two or more activities into one if the control proceeds onto the next activity irrespective of the path chosen.

An Activity Diagram using Merge Notation



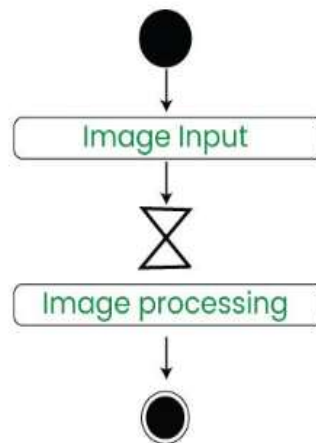
Merge

Activity diagrams

- This refers to an event that stops the flow for a time; an hourglass depicts it. We can have a scenario where an event takes some time to completed.

Let us assume that the processing of an image takes a lot of time. Then it can be represented as shown below.

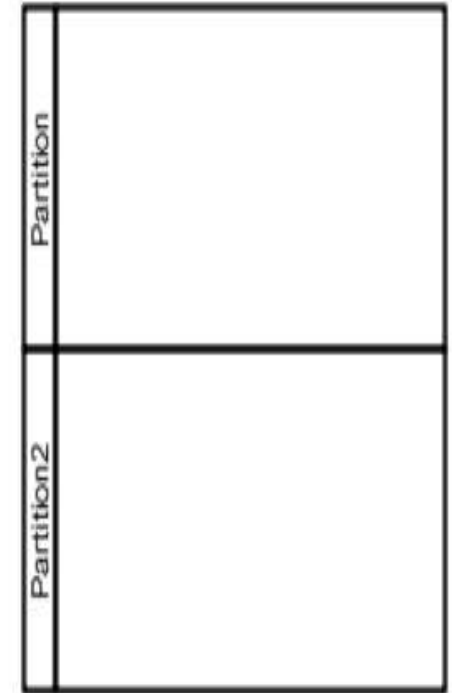
An Activity Diagram using Time Event Notation



Activity diagrams

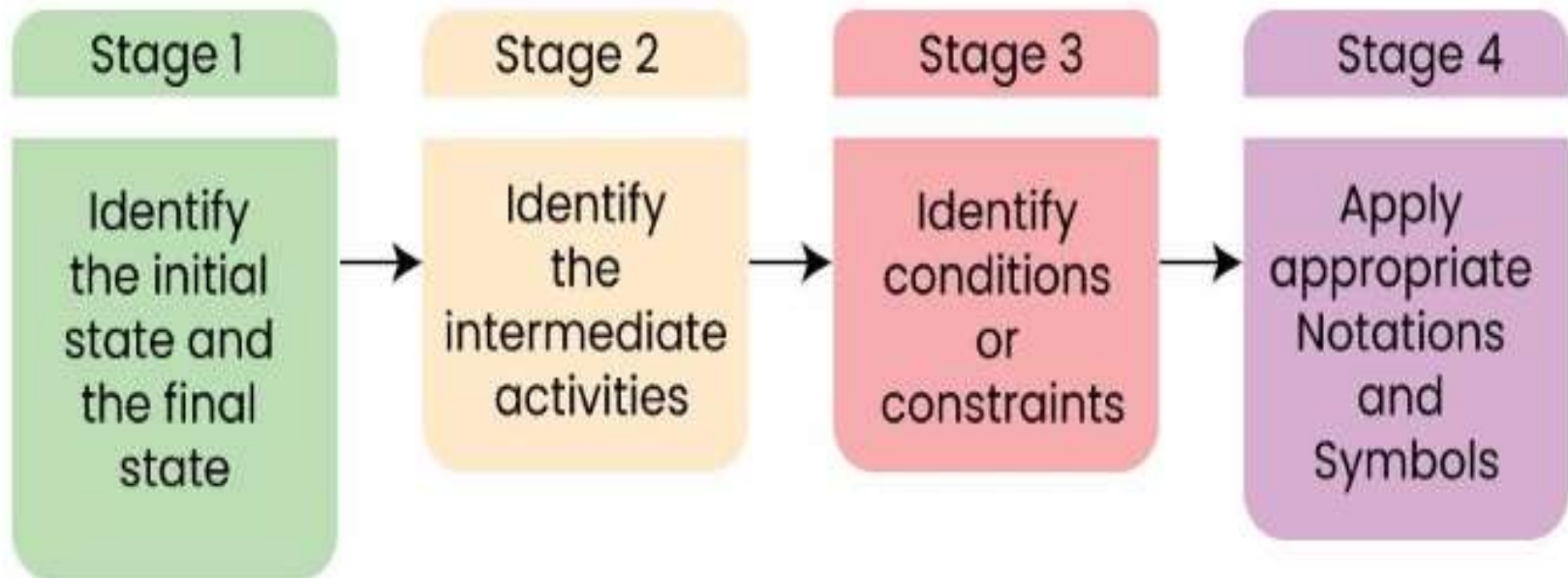
Swimlane and Partition

A way to group activities performed by the same actor on an activity diagram or to group activities in a single thread



Draw an Activity Diagram in UML

Steps to Draw an Activity Diagram



Draw an Activity Diagram in UML

Step 1. Identify the Initial State and Final States:

- This is like setting the starting point and ending point of a journey.
- Identify where your process begins (initial state) and where it concludes (final states).
- For example, if you are modeling a process for making a cup of tea, the initial state could be "No tea prepared," and the final state could be "Tea ready."

Step 2. Identify the Intermediate Activities Needed:

- Think of the steps or actions required to go from the starting point to the ending point.
- These are the activities or tasks that need to be performed.
- Continuing with the tea-making , intermediate activities could include "Boil water," "Pour tea into a cup".

Step 3. Identify the Conditions or Constraints:

- Consider the conditions or circumstances that might influence the flow of your process.
- These are the factors that determine when you move from one activity to another.
- Using the tea-making scenario, a condition could be "Water is boiled," which triggers the transition to the next activity.

Step 4. Draw the Diagram with Appropriate Notations:

- Now, represent the identified states, activities, and conditions visually using the appropriate symbols and notations in an activity diagram. This diagram serves as a visual map of your process, showing the flow from one state to another.

Draw an Activity Diagram in UML

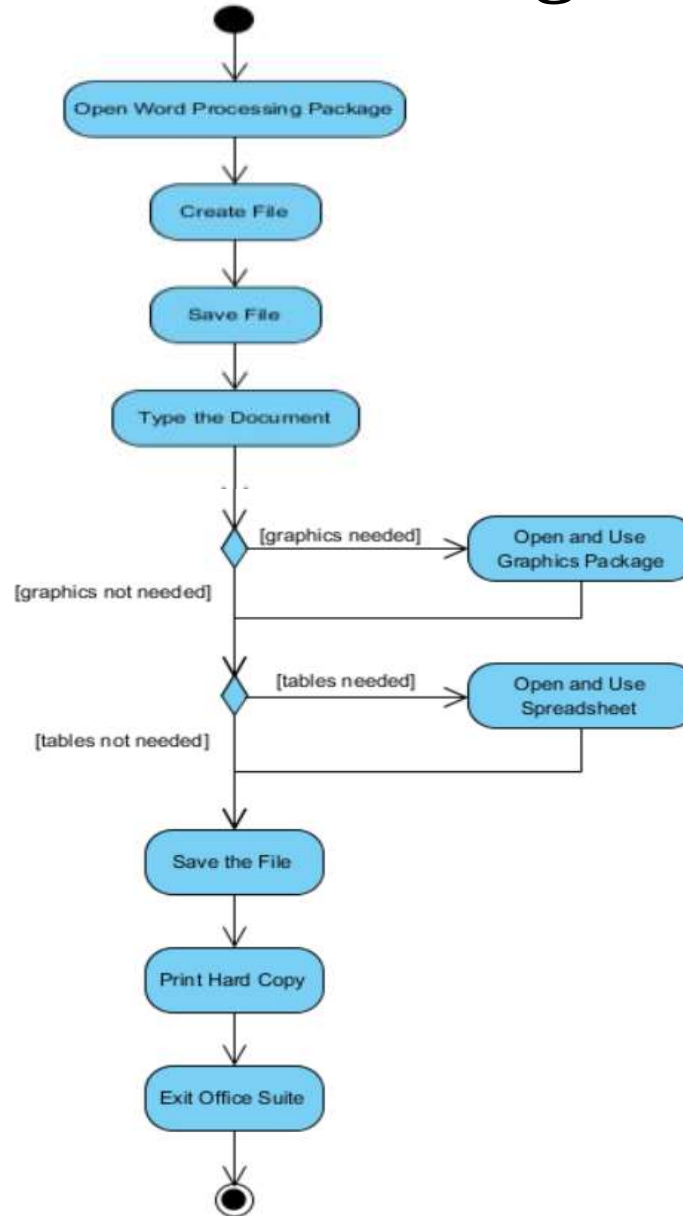
Activity Diagram Example

- Here's an example of an activity diagram that shows the process:

Steps:

- Open the word processing package.
- Create a file.
- Save the file under a unique name within its directory.
- Type the document.
- If graphics are necessary, open the graphics package, create the graphics, and paste the graphics into the document.
- If a spreadsheet is necessary, open the spreadsheet package, create the spreadsheet, and paste the spreadsheet into the document.
- Save the file.
- Print a hard copy of the document.
- Exit the word processing package.

Draw an Activity Diagram in UML



Activity Diagram Example - Process Order

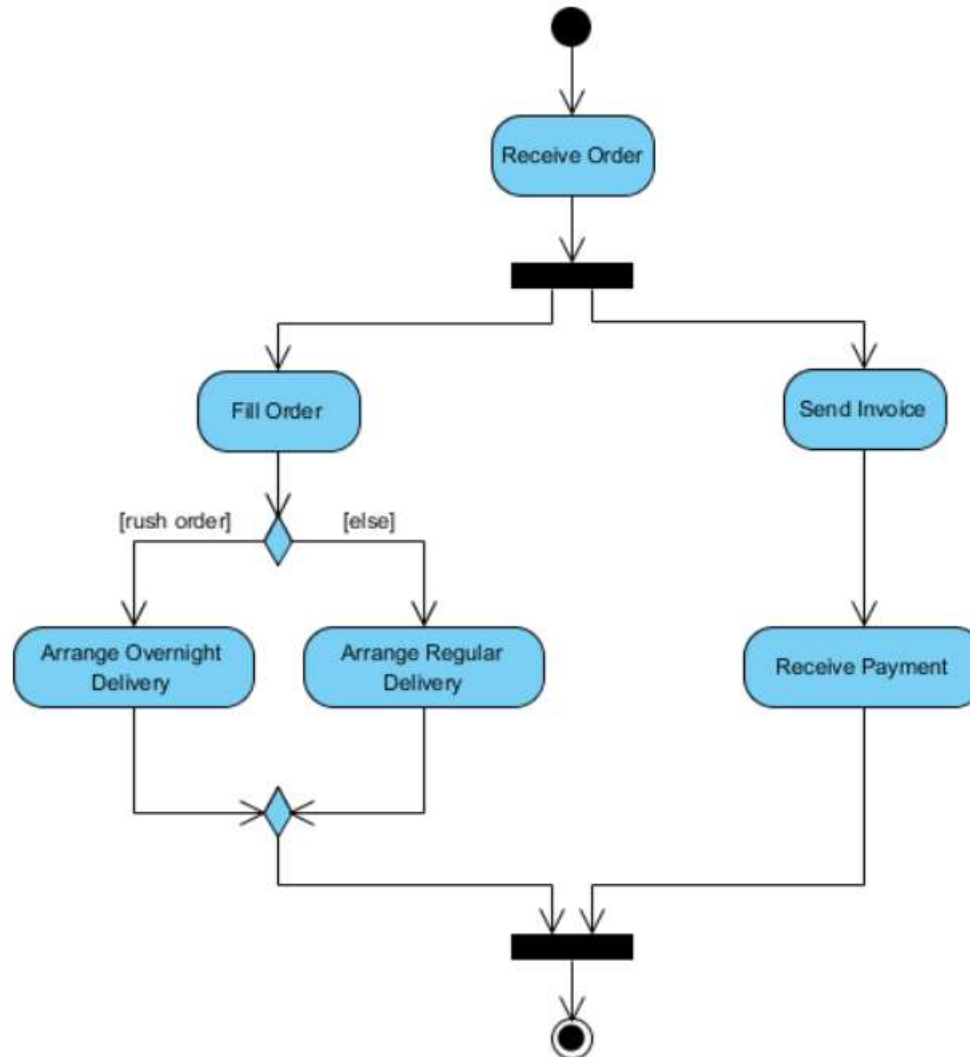
Process Order - Problem Description

Once the order is received, the activities split into two parallel sets of activities. One side fills and sends the order while the other handles the billing.

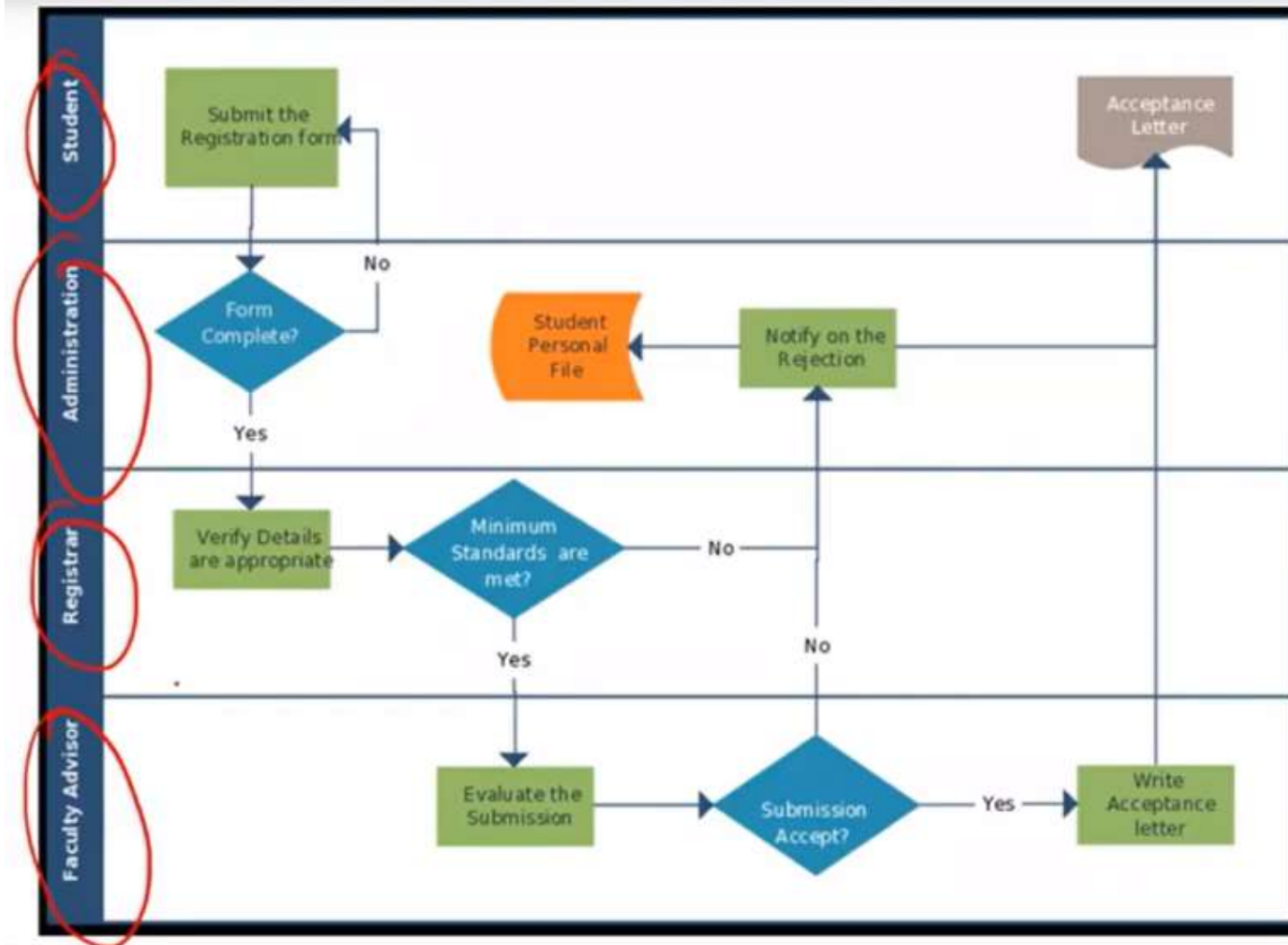
On the Fill Order side, the method of delivery is decided conditionally. Depending on the condition either the Overnight Delivery activity or the Regular Delivery activity is performed.

Finally the parallel activities combine to close the order.

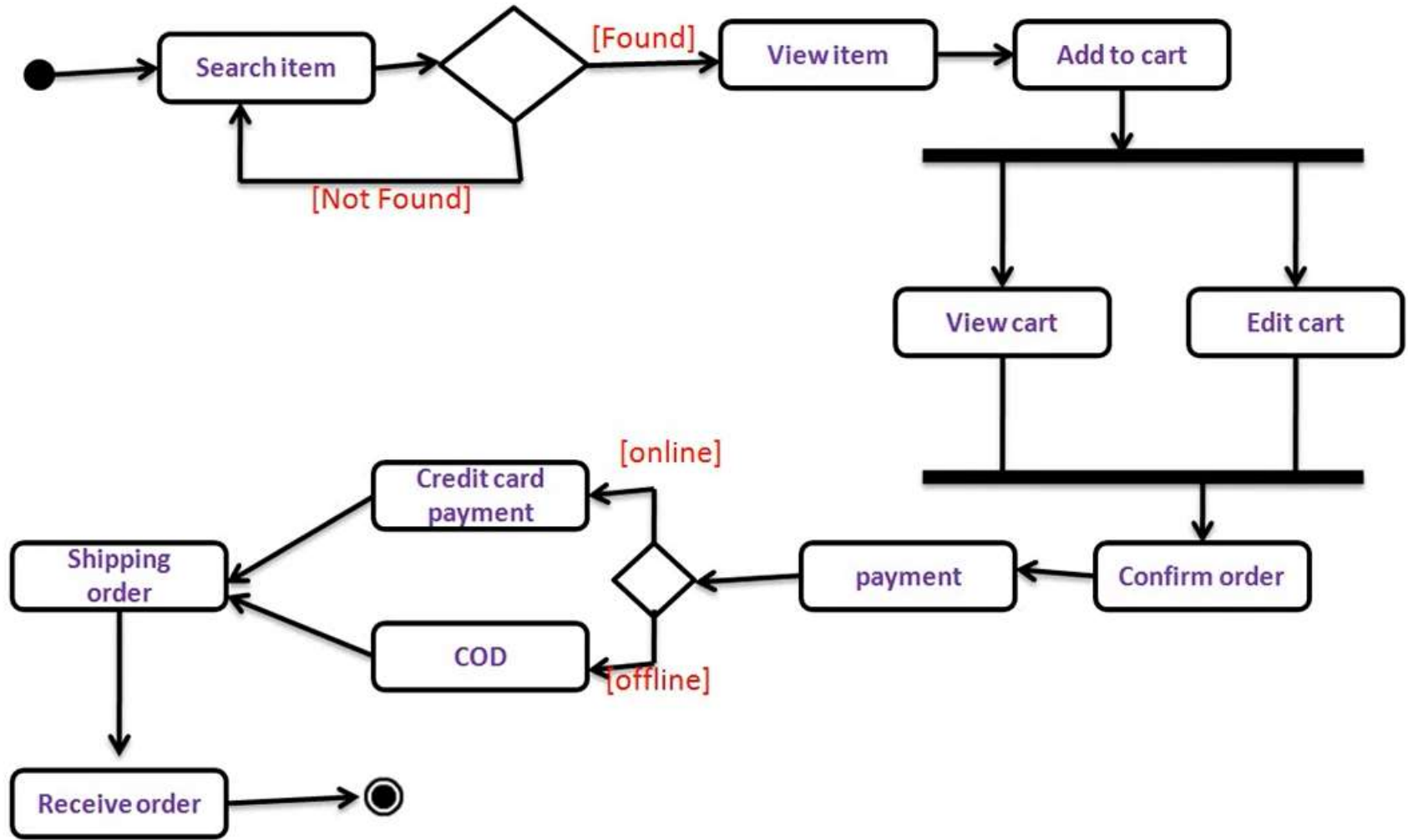
Activity Diagram Example - Process Order



Activity Diagram Example For- Swimlane



Activity Diagram Example - Online shopping system



Classwork

- **Activity Diagram for ATM machine System**

Use Case Diagram

- A **UML** use case diagram is the primary form of system/software requirements for a new software program underdeveloped.
- Use cases specify the expected behavior (what), and not the exact method of making it happen (how).
- Use cases once specified can be denoted both textual and visual representation (i.e. use case diagram).
- A key concept of use case modeling is that it helps us design a system from the end user's perspective.
- It is an effective technique for communicating system behavior in the user's terms by specifying all externally visible system behavior.
- **A use case diagram** is a visual representation used in software engineering to depict the interactions between **system actors** and the **system itself**.

A use case diagram is usually simple. It does not show the detail of the use cases:

- It only summarizes **some of the relationships** between use cases, actors, and systems.
- It does **not show the order** in which steps are performed to achieve the goals of each use case.

- **Purpose of Use Case Diagram**
- Specify the context of a system
- Capture the requirements of a system
- Validate a systems architecture
- Drive implementation and generate test cases
- Developed by analysts together with domain experts

- **Notation Description**

Actor: Named by noun.

- Actor plays a role in the business
- Similar to the concept of user, but a user can play different roles
- For example:
 - A prof. can be instructor and also researcher
 - plays 2 roles with two systems
- Actor triggers use case(s).
- Actor has a responsibility toward the system (inputs), and Actor has expectations from the system (outputs).



Use Case: Named by verb + Noun (or Noun Phrase).



- i.e. Do something
- Each Actor must be linked to a use case, while some use cases may not be linked to actors.

Communication Link:



- The participation of an actor in a use case is shown by connecting an actor to a use case by a solid link.
- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.

Boundary of system:

- The system boundary is potentially the entire system as defined in the requirements document.
- For large and complex systems, each module may be the system boundary.
- For example, for an ERP system for an organization, each of the modules such as personnel, payroll, accounting, etc.
- can form a system boundary for use cases specific to each of these business functions.
- The entire system can span all of these modules depicting the overall system boundary



System boundary is represented by a rectangular box.

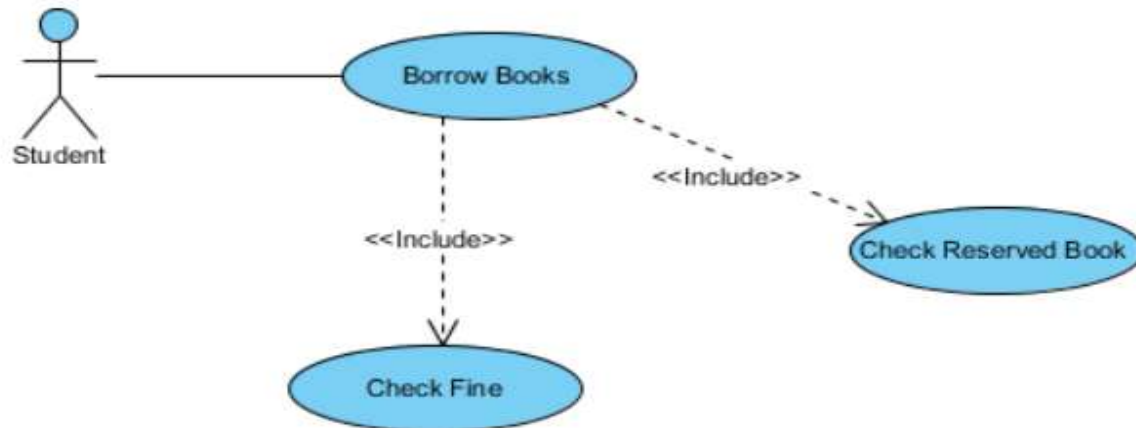
Use Case Example - Association Link

A Use Case diagram illustrates a set of use cases for a system, i.e. the actors and the relationships between the actors and use cases.



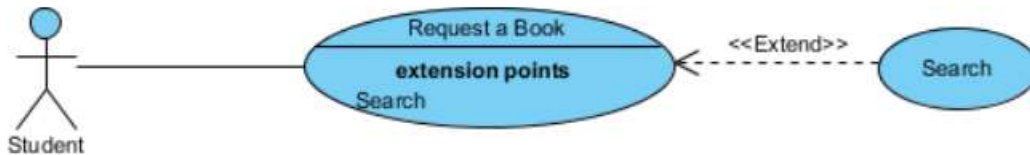
Use Case Example - Include Relationship

The include relationship adds additional functionality not specified in the base use case. The <<Include>> relationship is used to include common behavior from an included use case into a base use case in order to support the reuse of common behavior.



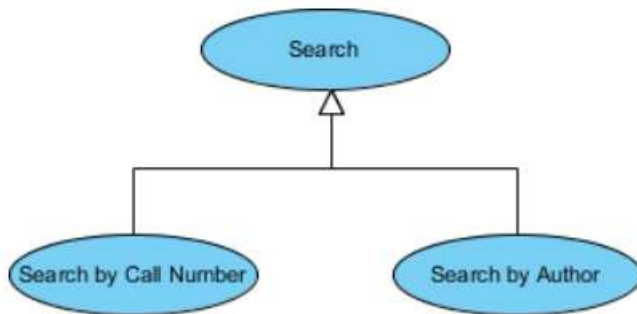
Use Case Example - Extend Relationship

The extend relationships are important because they show optional functionality or system behavior. The <<extend>> relationship is used to include optional behavior from an extending use case in an extended use case. Take a look at the use case diagram example below. It shows an extend connector and an extension point "Search".



Use Case Example - Generalization Relationship

A generalization relationship means that a child use case inherits the behavior and meaning of the parent use case. The child may add or override the behavior of the parent. The figure below provides a use case example by showing two generalization connectors that connect between the three use cases.



Draw a Use Case diagram in UML

Below are the main steps to draw use case diagram in UML:

- **Step 1: Identify Actors:** Determine who or what interacts with the system. These are your actors. They can be users, other systems, or external entities.
- **Step 2: Identify Use Cases:** Identify the main functionalities or actions the system must perform. These are your use cases. Each use case should represent a specific piece of functionality.
- **Step 3: Connect Actors and Use Cases:** Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.
- **Step 4: Add System Boundary:** Draw a box around the actors and use cases to represent the system boundary. This defines the scope of your system.
- **Step 5: Define Relationships:** If certain use cases are related or if one use case is an extension of another, you can indicate these relationships with appropriate notations.
- **Step 6: Review and Refine:** Step back and review your diagram. Ensure that it accurately represents the interactions and relationships in your system. Refine as needed.
- **Step 7: Validate:** Share your use case diagram with stakeholders and gather feedback. Ensure that it aligns with their understanding of the system's functionality.

How to Identify Actor ?

- Who uses the system?
- Who installs the system?
- Who starts up the system?
- Who maintains the system?
- Who shuts down the system?
- What other systems use this system?
- Who gets information from this system?
- Who provides information to the system?
- Does anything happen automatically at a present time?

How to Identify Use Cases?

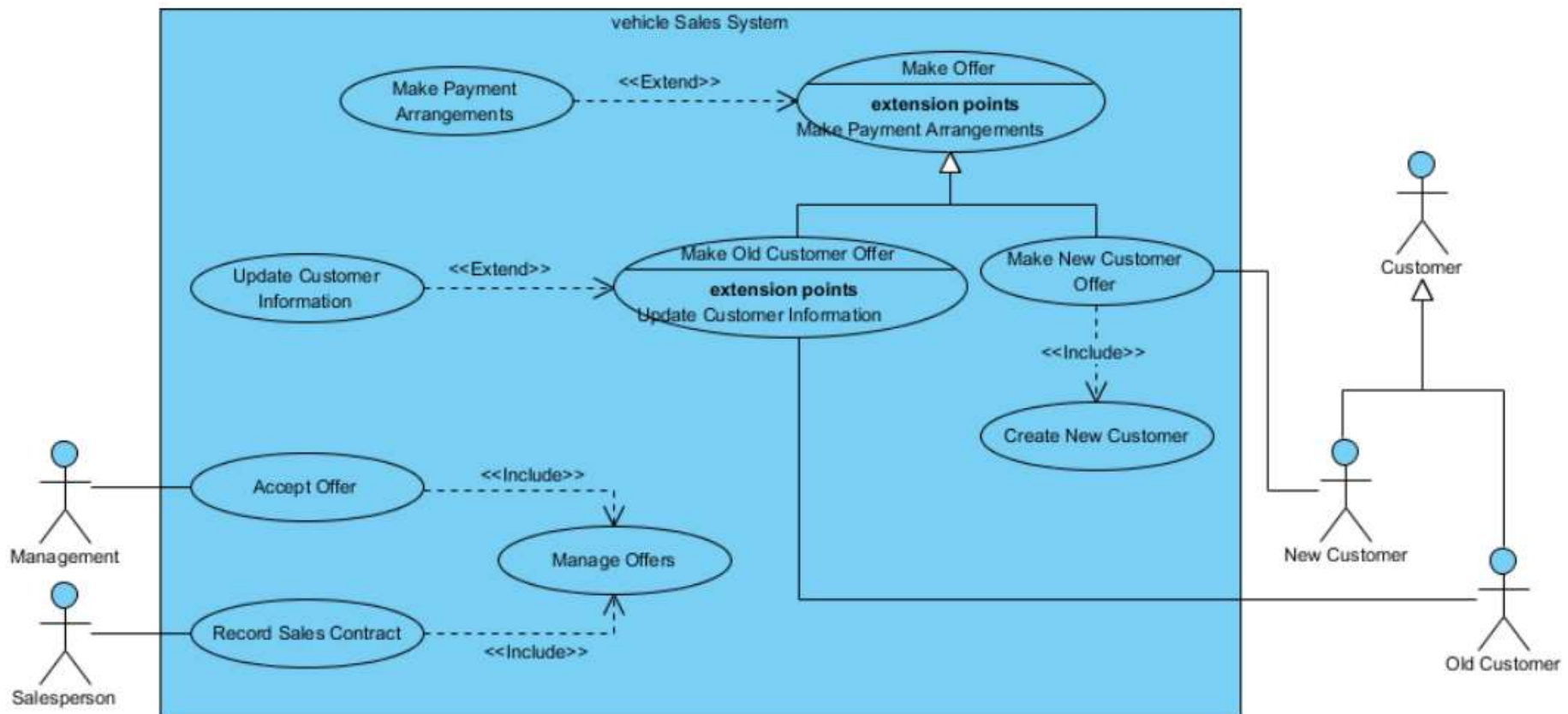
- What functions will the actor want from the system?
- Does the system store information? What actors will create, read, update or delete this information?
- Does the system need to notify an actor about changes in the internal state?
- Are there any external events the system must know about? What actor informs the system of those events?

Use Case Diagram Tips

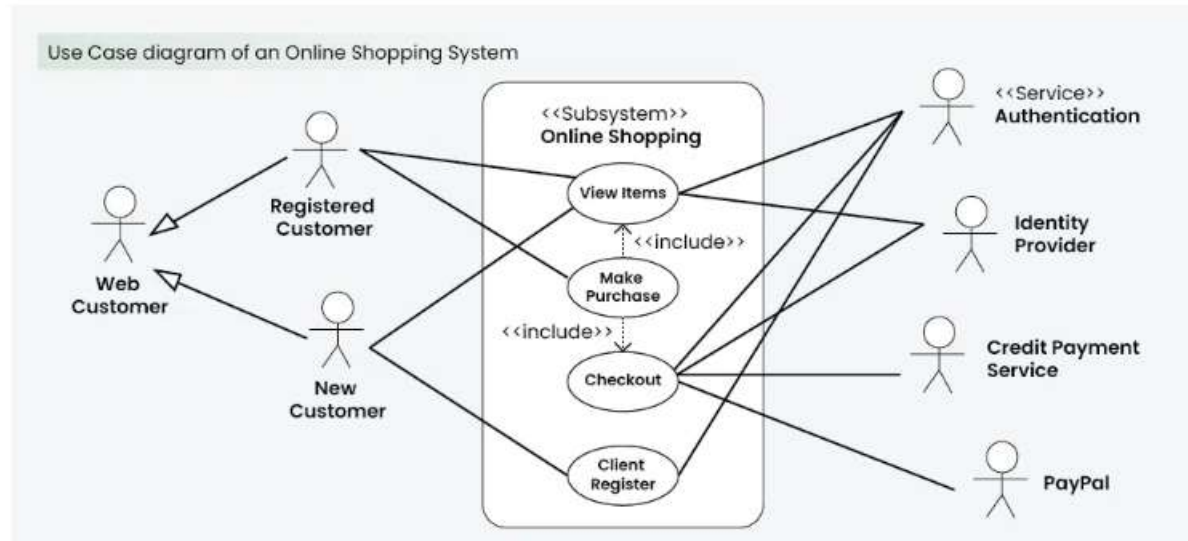
- Always structure and organize the use case diagram from the perspective of actors.
- Use cases should start off simple and at the highest view possible. Only then can they be refined and detailed further.
- Use case diagrams are based upon functionality and thus should focus on the "what" and not the "how".

Use Case Diagram - Vehicle Sales Systems

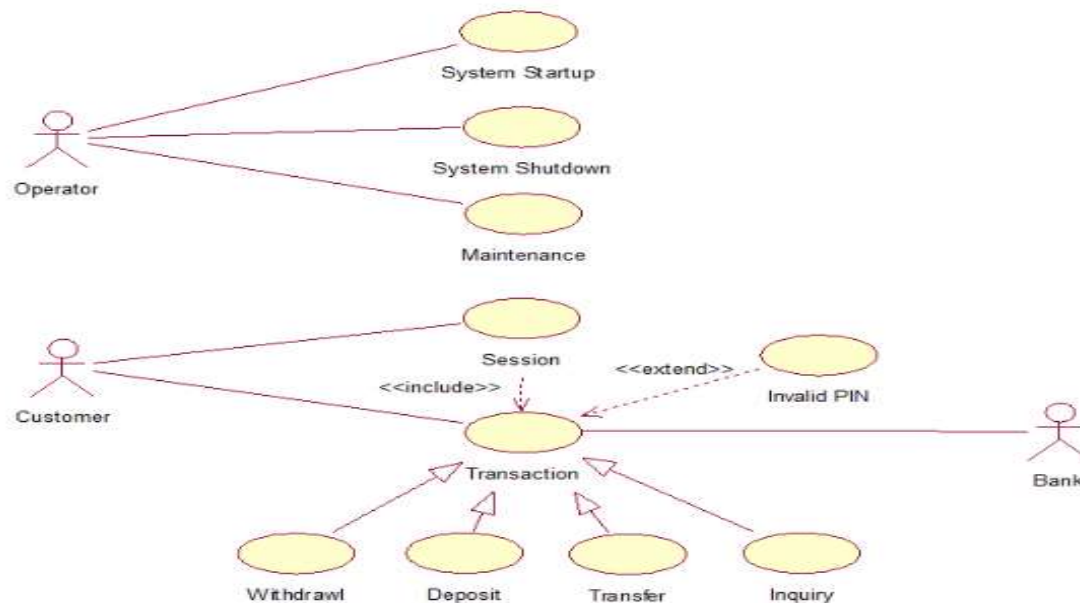
- The figure below shows a use case diagram example for a vehicle system. As you can see even a system as big as a vehicle sales system contains not more than 10 use cases! That's the beauty of use case modeling.
- The use case model also shows the use of extend and include. Besides, there are associations that connect between actors and use cases.



Below is the use case diagram of an Online Shopping System:



Use case diagram



Interaction diagrams

- Interaction diagrams in software engineering are Unified Modeling Language (UML) behavioral diagrams that visualize how objects in a system interact, communicate, and exchange messages to fulfill specific use cases.
- They model the dynamic behavior of systems, focusing on the flow of control and information over time.
- Key types include sequence diagrams (time-ordered messages) and communication/collaboration diagrams (structural relationships).

Types of Interaction Diagrams

- **Sequence Diagram:** Emphasizes the time ordering of messages sent between objects. It is the most common interaction diagram, reading top-to-bottom.
- **Communication (Collaboration) Diagram:** Focuses on the structural organization of objects and their relationships, highlighting which objects are connected.
- **Interaction Overview Diagram:** A hybrid of activity and sequence diagrams, providing a high-level view of how smaller interactions fit together within a system.
- **Timing Diagram:** Focuses on exact timing constraints on messages and state changes.

Sequence Diagram

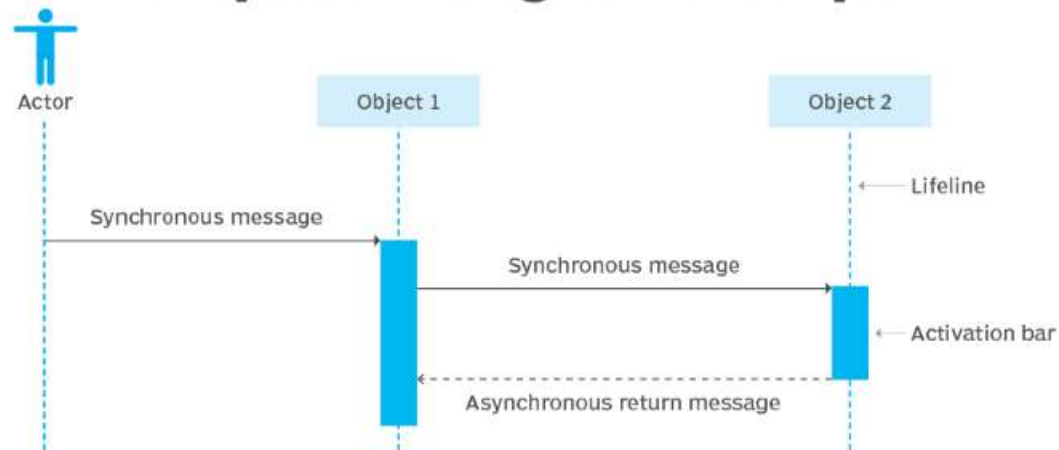
- A sequence diagram is a Unified Modeling Language (UML) diagram that visualizes interactions between objects in a specific sequential order over time.
- It shows how actors and objects communicate via messages to perform processes, such as use cases, acting as a key tool for developers to model system behavior, design architecture, and document workflows.
- **A sequence diagram consists of :-**
- **Participants:** This could be an actor, an entity, a database, or a boundary, among others.
- The different types of participants each have their own symbol to represent them.
- And each participant appears with a vertical bar to represent the passage of time.
- **Messages:** These are horizontal arrows between participants that show the interactions between participants, such as data exchanges or method calls. Typically they're labeled.
- Messages are ordered from top to bottom to show what happens, from first to last.

Creating a Sequence Diagrams

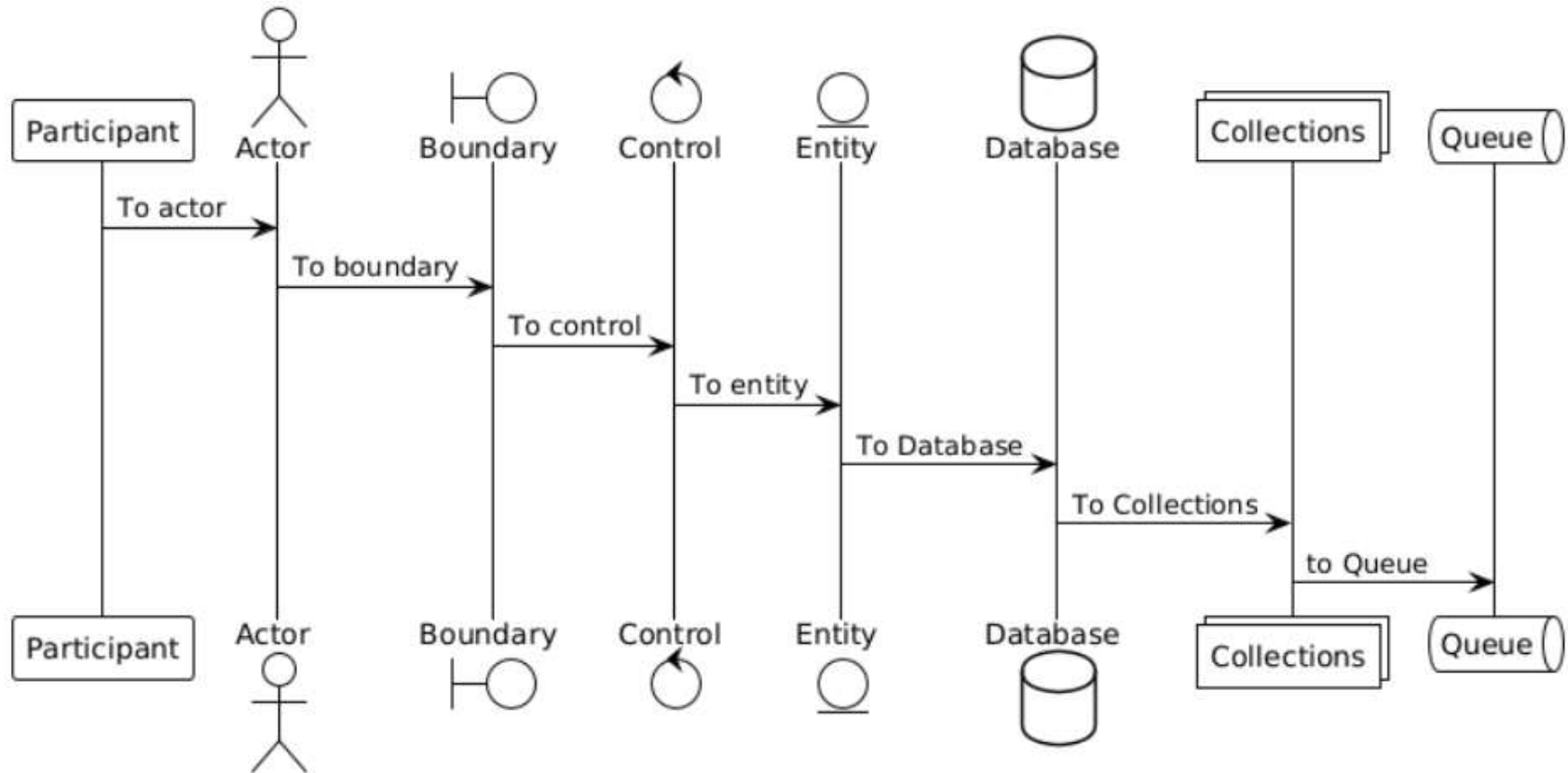
- Creating a sequence diagram involves several steps, and it's typically done during the design phase of software development to illustrate how different components or objects interact over time. Here's a step-by-step guide on how to create sequence diagrams:
- **Step 1: Identify the Scenario:** Understand the specific scenario or use case that you want to represent in the sequence diagram. This could be a specific interaction between objects or the flow of messages in a particular process.
- **Step 2: List the Participants:** Identify the participants (objects or actors) involved in the scenario. Participants can be users, systems, or external entities.
- **Step 3: Define Lifelines:** Draw a vertical dashed line for each participant, representing the lifeline of each object over time. The lifeline represents the existence of an object during the interaction.
- **Step 4: Arrange Lifelines:** Position the lifelines horizontally in the order of their involvement in the interaction. This helps in visualizing the flow of messages between participants.
- **Step 5: Add Activation Bars:** For each message, draw an activation bar on the lifeline of the sending participant. The activation bar represents the duration of time during which the participant is actively processing the message.
- **Step 6: Draw Messages:** Use arrows to represent messages between participants. Messages flow horizontally between lifelines, indicating the communication between objects. Different types of messages include synchronous (solid arrow), asynchronous (dashed arrow), and self-messages.
- **Step 7: Include Return Messages:** If a participant sends a response message, draw a dashed arrow returning to the original sender to represent the return message.
- **Step 8: Indicate Timing and Order:** Use numbers to indicate the order of messages in the sequence. You can also use vertical dashed lines to represent occurrences of events or the passage of time.

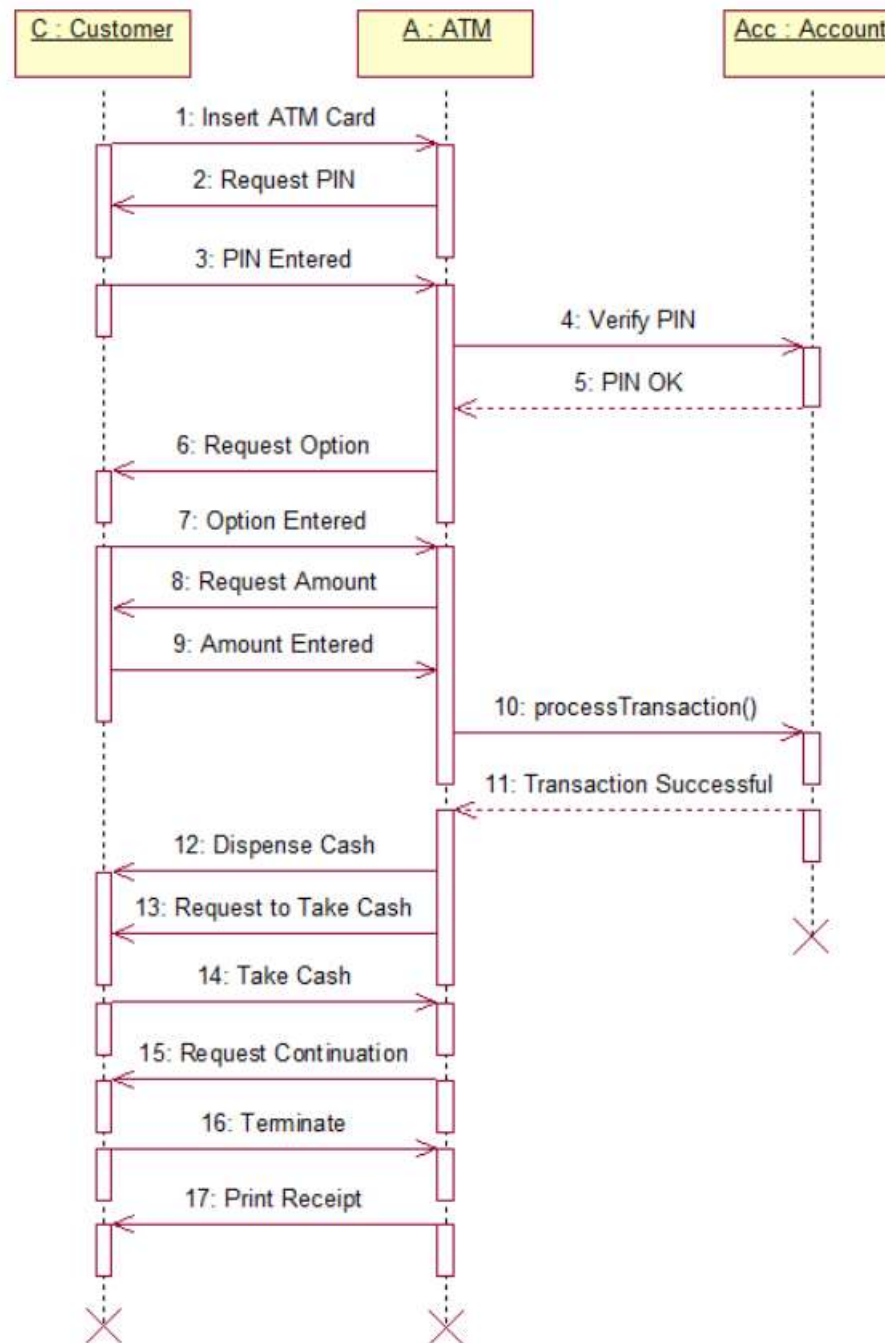
- **Step 9: Include Conditions and Loops:** Use combined fragments to represent conditions (like if statements) and loops in the interaction. This adds complexity to the sequence diagram and helps in detailing the control flow.
- **Step 10: Consider Parallel Execution:** If there are parallel activities happening, represent them by drawing parallel vertical dashed lines and placing the messages accordingly.
- **Step 11: Review and Refine:** Review the sequence diagram for clarity and correctness. Ensure that it accurately represents the intended interaction. Refine as needed.
- **Step 12: Add Annotations and Comments:** Include any additional information, annotations, or comments that provide context or clarification for elements in the diagram.
- **Step 13: Document Assumptions and Constraints:** If there are any assumptions or constraints related to the interaction, document them alongside the diagram.
- **Step 14: Tools:** Use a UML modeling tool or diagramming software to create a neat and professional-looking sequence diagram. These tools often provide features for easy editing, collaboration, and documentation.

Sequence diagram example

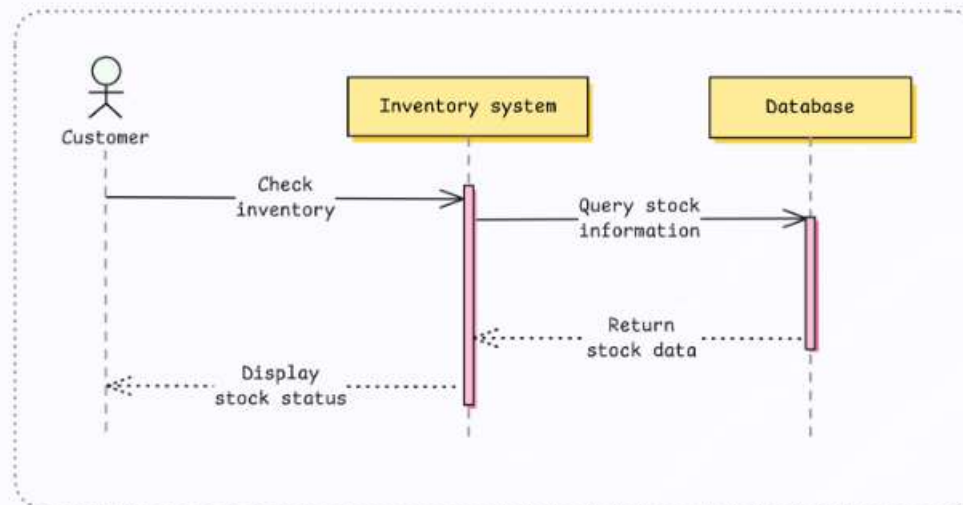
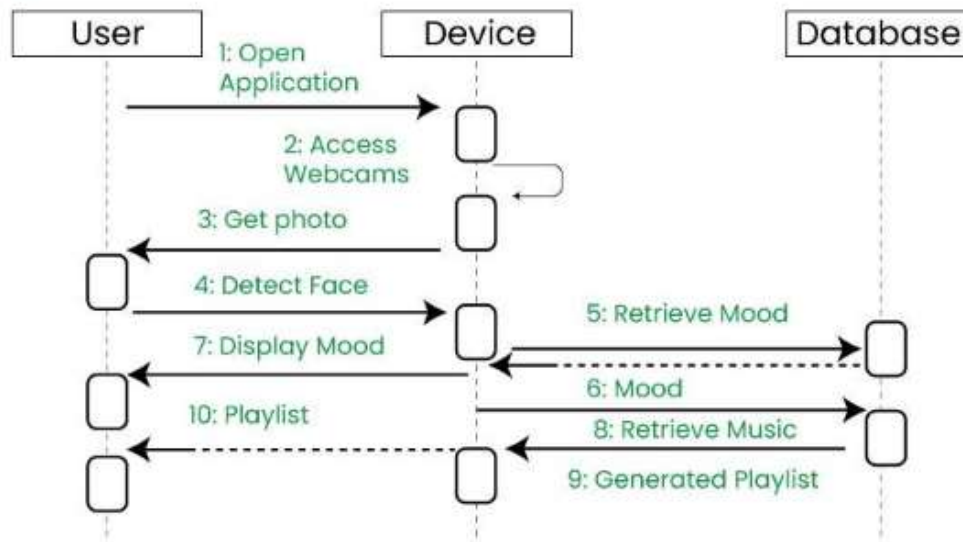


Sequence Diagram Participants



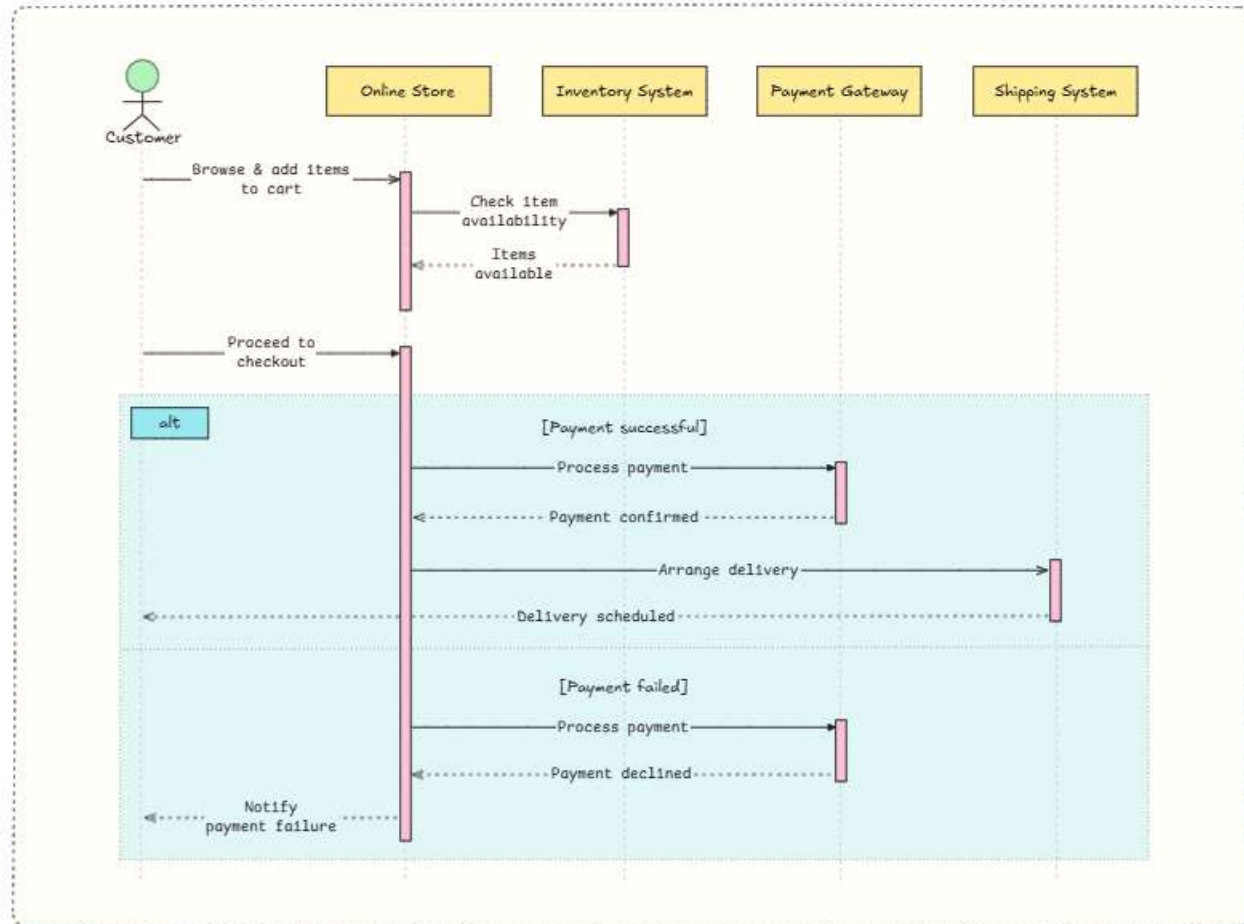


Example sequence diagram



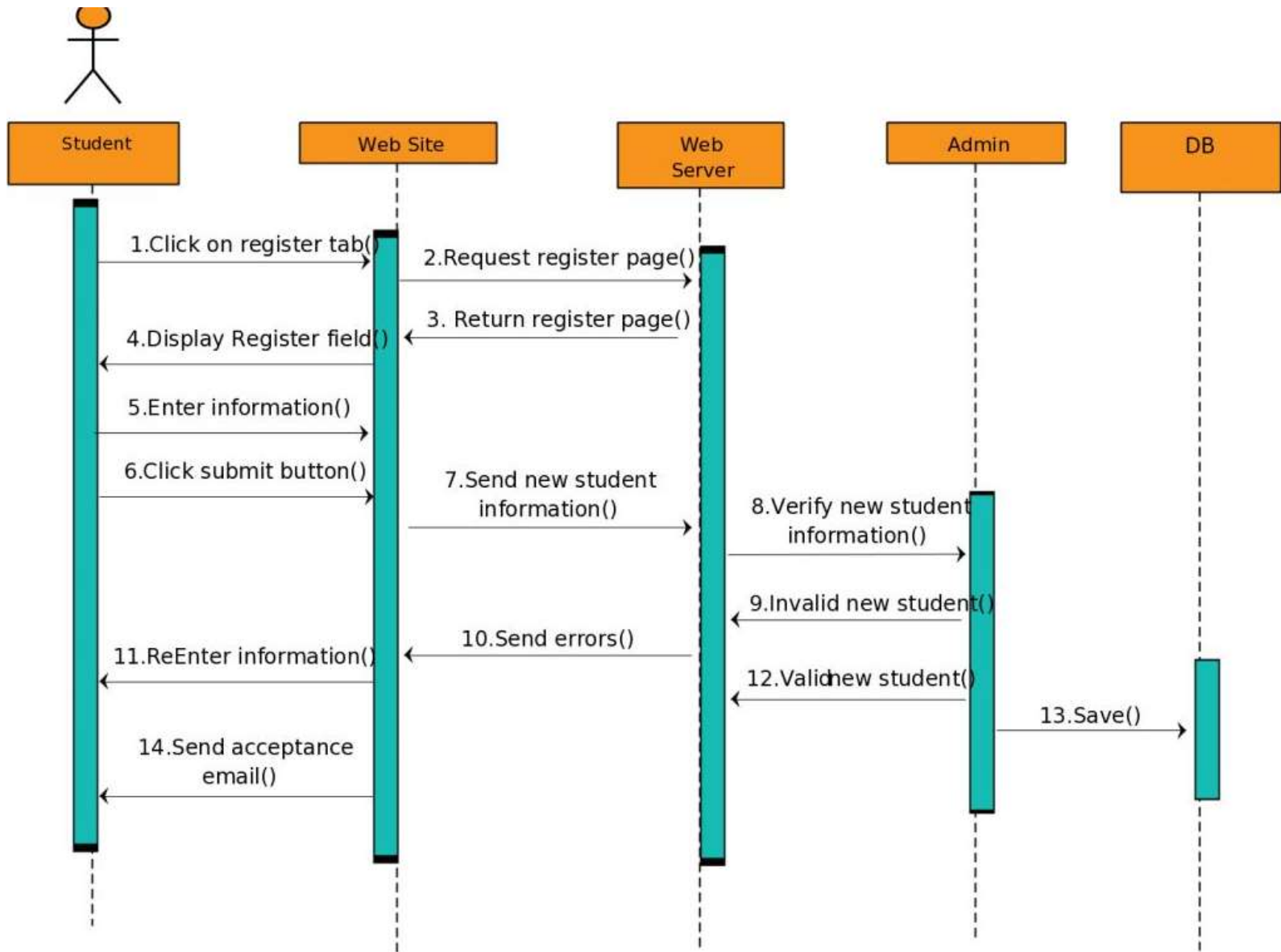
Sequence diagram for an online order 🛒

and zoom in!



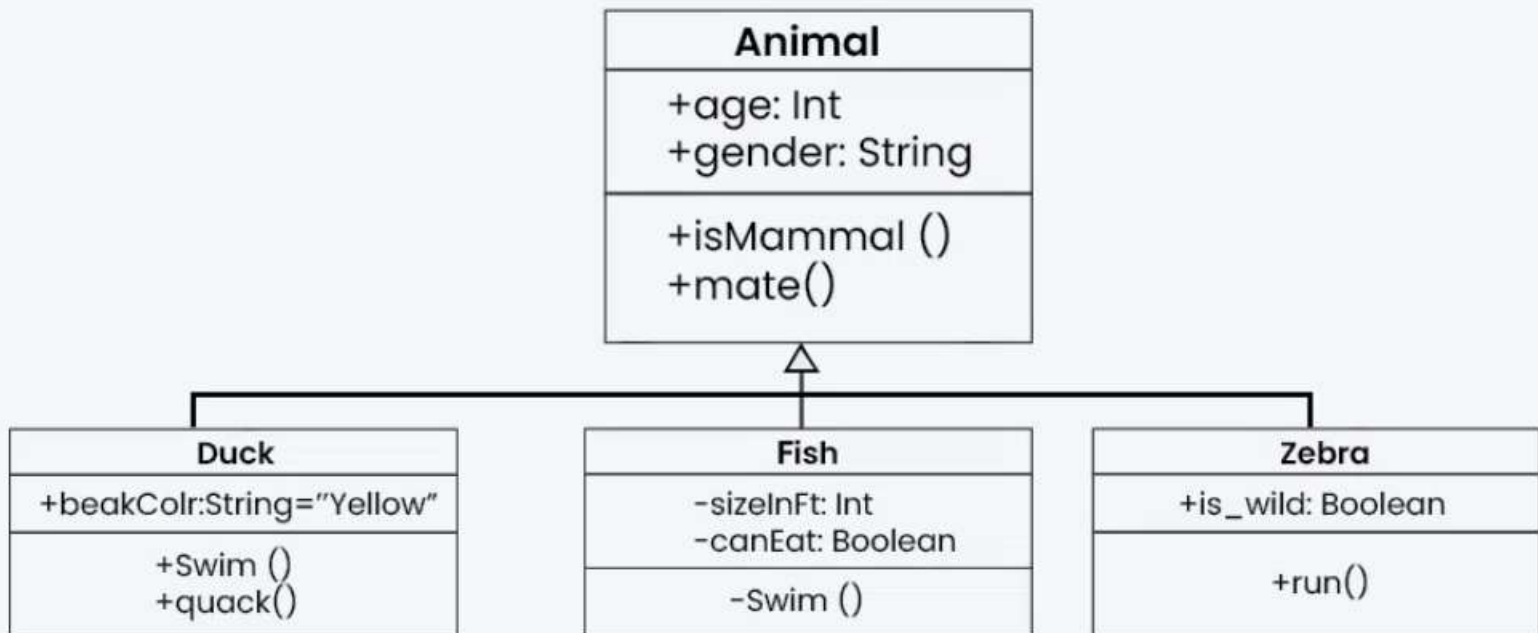
- **Classwork Example:**

1. **Student Registration Sequence Diagram**
2. **Library Book Borrowing Sequence Diagram**

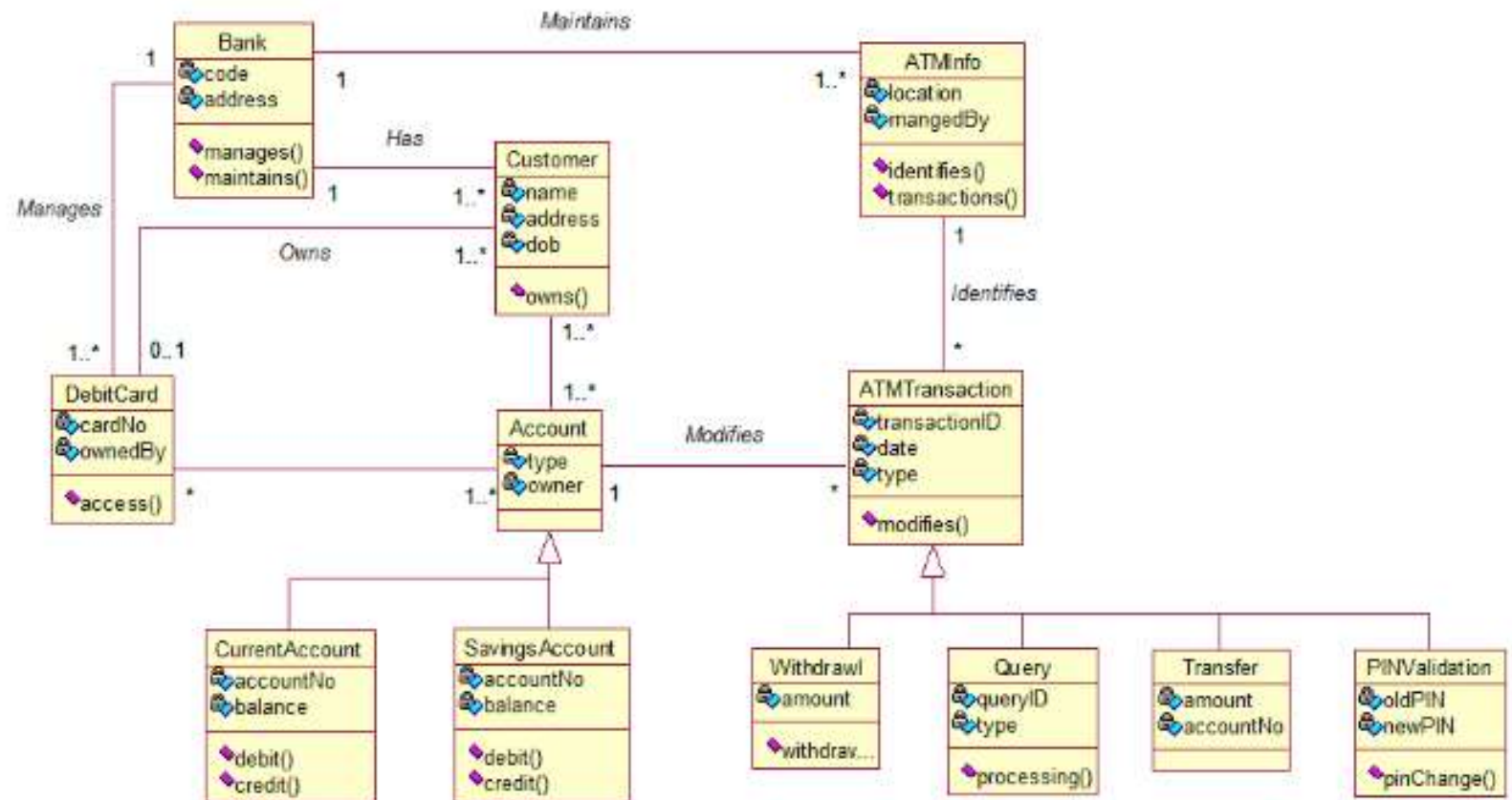


Class Diagram

- A UML class diagram shows the structure of a system by displaying its classes, their attributes, methods, and the relationships between them.
- It helps the team understand how the system is organized and how components interact.
- Represents classes, attributes, methods, and relationships
- Helps communicate and document the software structure
- Makes it easier for developers and designers to understand the system

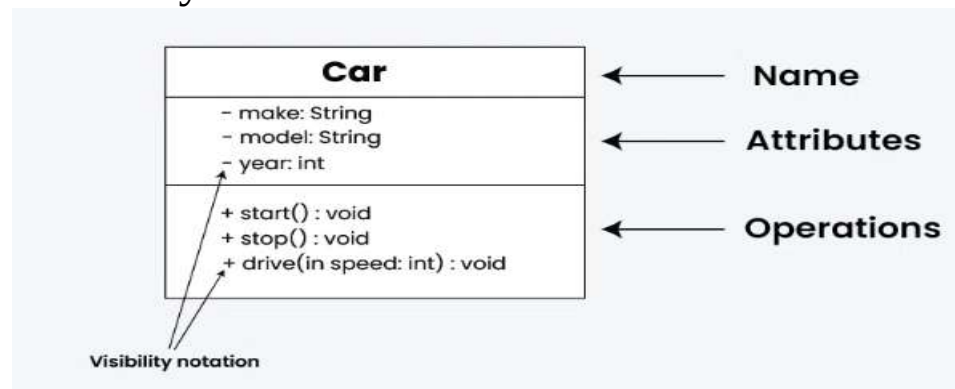


Class diagram



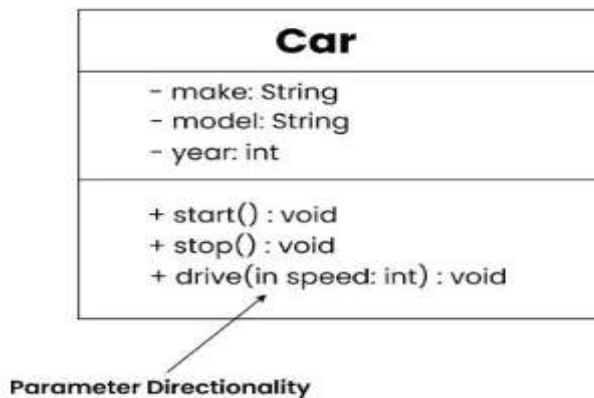
UML Class Notation

- Classes are depicted as boxes, each containing three compartments for the class name, attributes, and methods.
- **Class Name:** The name of the class is typically written in the top compartment of the class box and is centered and bold.
- **Attributes:** Attributes, also known as properties or fields, represent the data members of the class. They are listed in the second compartment of the class box and often include the visibility (e.g., public, private) and the data type of each attribute.
- **Methods:** Methods, also known as functions or operations, represent the behavior or functionality of the class. They are listed in the third compartment of the class box and include the visibility (e.g., public, private), return type, and parameters of each method.
- **Visibility Notation:** Visibility notations indicate the access level of attributes and methods. Common visibility notations include:



UML Class Notation

- **Parameter Directionality**
- In class diagrams, parameter directionality refers to the indication of the flow of information between classes through method parameters.
- It helps to specify whether a parameter is an input, an output, or both. This information is crucial for understanding how data is passed between objects during method calls.
- There are three main parameter directionality notations used in class diagrams:
- **In (Input):** A parameter sent from the caller to the method being executed.
- **Out (Output):** A parameter sent from the method back to the caller after execution.
- **InOut (Input and Output):** A parameter that is sent to the method and also returns back to the caller.



Benefits of Class Diagrams

- Class diagrams represent the system's classes, attributes, methods, and relationships, providing a clear view of its architecture.
- They show various relationships between classes, such as associations and inheritance, helping stakeholders understand component connectivity.
- Class diagrams serve as a visual tool for communication among team members and stakeholders, bridging gaps between technical and non-technical audiences.
- They guide developers in coding by illustrating the design, ensuring consistency between the design and actual implementation.
- Many development tools allow for code generation from class diagrams, reducing manual errors and saving time.